SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: Theory of Computation **COURSE CODE: CSA1301**

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

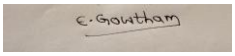
Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:50**
Duration : 1 Hour

Application Based Question

Code of Conduct:

I Enamala Gowtham /192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

1.In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable.Design a **Deterministic Finite Automaton (DFA)** over the alphabet Σ = {0,1} to validate passwords that **end with the pattern "01"**.Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

2.In web applications, validating user input is essential to ensure correctness, security, and performance.Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet Σ = {a, b}.Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications**.**

3.In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role.Demonstrate how the regular expression `a*b*` can be converted into an equivalent finite automaton (FA).Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

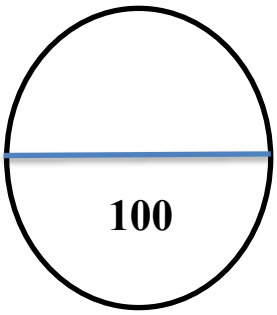
4.In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language L= { $a^n b^n$ | n ≥ 0 } cannot be recognized by any finite automaton.Further, analyze the implications of this limitation in the design of modern compilers.

5.Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules.Explain how the **closure properties of regular languages** enable the combination of multiple detection rules.Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50



2. Can enforce rules like:

- **Email format**
- **Password rules**
- **Input structure**

CO2 Assessment-1

Name : E. Gowtham

Reg.NO : 192311190

Course : Theory of Computation

code : CSA 1301

1) Design and minimize a DFA for binary strings ending with 01

Q]: Design a DFA over $\Sigma = \{0,1\}$ that accepts strings ending with 01. Minimize the DFA and explain its efficiency in Real world hardware.

A Deterministic Finite Automaton (DFA) is a finite state machine in which each state has exactly one transition for every input symbol.

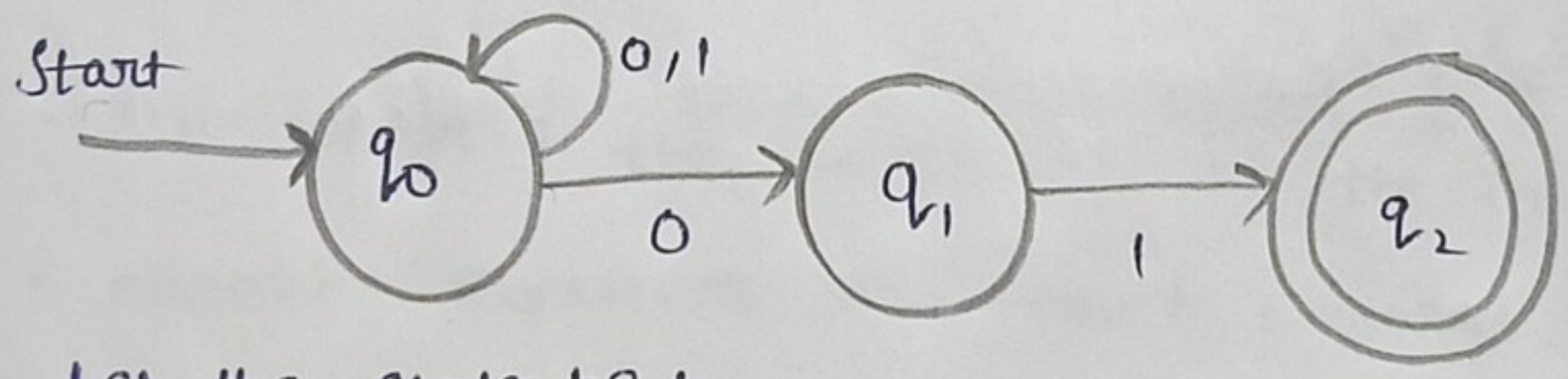
The language consists of all binary strings whose last two symbols are 01.

Examples:

- Accepted: 01, 101, 0001, 1101
- Rejected: 0, 1, 10, 011, 111

State Diagram:

Example - $L = \{w \mid w \text{ ends with } 01\}$



Let the states be;

q_0: No useful final 0 is currently matched; intial state

q_1: The most recent Symbol is 0.

q_2: The string currently ends with 01; accepting state.

Transition Table

Current state	Input 0	Input 1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

Minimization:

The DFA is already minimal with three states. Each state represents a distinct situation needed to determine whether the input ends with 01. No two states can be merged without changing the accepted language.

Real-world efficiency:

A minimized DFA requires only three states and two outgoing transitions per state. It can be implemented using a small number of hardware Registers and simple logic gates. This Reduces memory usage, circuit complexity, and processing overhead in embedded systems and authentication modules.

Conclusion:

The minimal DFA has three states and accepts exactly those binary strings that end with 01.

2] write a Regular expression for the pattern aaaabb,

Q: Design a Regular expression for the exact input pattern aaaabb over $\Sigma = \{a, b\}$, and explain its application in web applications.

A Regular expression describes a pattern that a string must follow.

The required Pattern contains exactly four a symbols followed by exactly two b symbols.

Regular expression:

aaaabb

without a space, The expression is

aaaabb

If the expression must match the entire input string, use the anchored form:

</> Regex

^aaaabb$

- ^ represents the Beginning of the input
- aaaabb Represents the exact Required sequence
- $ Represents the end of the input.

Examples:

Input	Result
aaaabb	Accepted
aaaab	Rejected
aaaaabb	Rejected
aaaabb	Rejected
aabbaa	Rejected

Real world applications:

Regular expressions are used in web applications to validate email formats, password rules, phone numbers, postal codes, and structured user inputs. They help identify invalid input before it is processed further.

Efficiency:

Regular patterns can be checked using Regex engines or equivalent finite automata. This makes validation efficient and helps maintain consistency in input checking. However, actual security also requires proper server-side validation and other protections.

Conclusion:

The anchored regular expression ^aaaabb$ accepts only the exact string aaaabb.

3) Convert the Regular a^*b^* into an equivalent finite automation

The Regular expression a^*b^* represents zero or more occurences of a, followed by zero or more occurences of b.

Here:

- a^* means zero or more a symbols
- b^* means zero or more b symbols
- The string may contain only a symbols, only b symbols, or both in that order.
- The empty string ϵ is also accepted.

Examples of Rejected accepted strings: ba, aba, baba

Finite automation Construction:

We can Construct a DFA with three states

- q_0 : initial state; reads zero or more a symbols.
  q_1: Reads b symbols after the a phase
  q_d: Dead state; entered if an a occurs after a b.

The accepting states are q_0 and q_1.

Transition table

State	Input a.	Input b
$\rightarrow *q_0$	q_0	q_1
$*q_1$	q_d	q_1
q_d	q_d	q_d

Here q_d is non-accepting dead state.

Why are regular expressions and finite automata equivalent?

Regular expressions and finite automata are equivalent because both describe exactly the class of regular languages.

- Every regular expression can be converted to into an equivalent finite automation
- Every finite automation can be converted into an equivalent Regular expression.
- Both Recognize the same set of strings, even though their representations differ.

Real world applications:

These concepts are used in lexical analysis in compilers intrusion detection systems, log monitoring, and text searching.

Conclusion:

The regular expression $a^* b^*$ can be recognized by a finite automation, demonstration the equivalence of regular expressions and finite automata.

04) use the pumping lemma to prove that $L = \{a^n b^n \mid n \geq 0\}$ is not regular.

The language is

$$L = \{a^n b^n \mid n \geq 0\}$$

It contains strings with an equal number of a symbols followed by b symbols.

Examples: ϵ, ab, aabb, aaabbb, aaaabbbb.

The pumping lemma is used to prove that certain languages are not regular.

Proof using the pumping lemma

Assume that L is regular

According to the pumping lemma,

$$S = xyz$$

The following conditions must hold:

1. $|xy| \leq p$.
2. $|y| \geq 0$
3. $xy^i z \in L$ for every $i \geq 0$

Step 1: choose a string

Choose the string

$$S = a^p b^p$$

The string belongs to L, because it contains p occurence of a followed by p occurences of b.

Step 2: Apply the pumping lemma

Since $|xy| \leq p$, the parts x and y occur entirely within the first p symbols, which are all a

Therefore, $y = a^k$, where $k \geq 1$.

Step 3: pump down

Choose $i = 0$, Then

$$xy^0z = xz = a^{p-k}b^p$$

This string containg fewer a symbols than b symbols. Therefore, it does not belong to L.

Implications of compiler design

finite automata cannot Recognize languages that Require an arbitary comparision of two counts such as equal numbers of a and b symbols in the specified order.

compilers use finite automata for lexical analysis, such as Recognizing identifiers, keywords, and numeric tokens. for nested structure and other context - free patterns, compilers need more powerful mechanisms, such as context — free grammars and pushdown automata

Conclusion:

The pumping lemma proves that $L = \{a^n b^n \mid n \geq 0\}$ is not regular because a finite automaton cannot Remember an unbounded number of a symbols to compare against the number of b symbols.

6) Explain the closure properties of Regular languages and their applications

Closure properties describe whether a class of languages remains within the same class after applying certain operations.

Regular languages are closed under several operations. This means that when regular languages are combined using these operations, the resulting language is also regular.

Important closure properties

operation	meaning	Example
union	strings belonging to either language	$L_1 \cup L_2$
Intersection	strings common to both languages	$L_1 \cap L_2$
Complement	strings not belonging to a language	$\overline{L_1}$
Difference	strings in one language but not the other	$L_1 - L_2$
Concatenation	A string from one language followed by one from another	$L_1 L_2$
Kleene star	Zero or more Repetions of a language	L_1^*

Example:

Let

$$L_1 = \{a\}, \quad L_2 = \{b\}$$

Both languages are Regular

1. union

$$L_1 \cup L_2 = \{a, b\}$$

The Resulting language is Regular

2. Concatenation

$$L_1 L_2 = \{ab\}$$

The Resulting language is also Regular

3. kleene star:

$$L_1^* = \{\epsilon, a, aa, aaa, \ldots\}$$

This language is Regular because it can be Recognized by a finite automaton.

4. Intersection

$$L_1 \cap L_2 \neq \emptyset$$

The empty language is Regular, so the interaction is Regular as well

Conclusion:

Closure properties allows multiple Regular language rules to be combined while preserving regularity. The Resulting languages can still be Recognized by finite automata, enabling efficient pattern matching in practical systems.